Gas Optimisation

The `revealAndDraw` function was identified as the most computationally expensive function and was specifically targeted for optimisation.

Before vs After (`revealAndDraw`)

Metric	`lottery.sol` (Before)	`gasOptimisedLottery.sol` (After)	Savings
Avg	44,121	44,031	90 gas

Techniques Applied

- **Struct packing** — The `Round` struct is ordered so that `address winner` (20 bytes), `Phase phase` (1 byte), and `bool prizeClaimed` (1 byte) are packed into a single 32-byte storage slot. Without this packing, each field would occupy its own slot, costing an extra SLOAD (2,100 gas cold / 100 gas warm) per additional slot accessed. This is a design-time optimisation applied in both contracts.
- `unchecked` **block on modulo** — The Solidity 0.8+ compiler injects a zero-divisor check before every `%` operation. Since `totalParticipants > 0` is already guaranteed by the `closeSale()` phase guard, wrapping the modulo in `unchecked {}` eliminates the redundant safety opcodes.
- **Explicit memory caching** — State variables (`targetBlock`, `winner`, `prizePool`) are read from storage once into local stack variables, avoiding repeat warm SLOADs (100 gas each) and preventing stack-shuffling penalties introduced by the `unchecked` scope.

Per-Technique Gas Impact

#	Technique	Target	Gas Saved
1	Struct packing	`Round` struct (slot 0)	Up to 2,100 per avoided cold SLOAD
2	`unchecked` modulo	`winnerIndex` calculation	∼35 gas (removes ISZERO + JUMPI opcodes)
3	Explicit memory caching	`targetBlock`, `winner`, `prizePool`	∼55 gas (avoids repeat warm SLOADs)
Total measured delta (avg)			**90 gas**